

ONDO 구조 노트

서비스의 개념, 구조도, AI 두 개와 추천 엔진이 실제로 어떻게 계산하는지를 저장소 코드 (10/2 버전) 기준으로 정리했습니다. 위에서부터 순서대로 읽으면 발표 질문에 답할 수 있는 수준까지 올라가도록 구성했습니다.

01 전체 그림

한 문장으로: 사진과 설문으로 퍼스널컬러·골격을 진단하고, 오늘 날씨에 안 맞는 옷을 빼고, 나에게 맞는 순서로 정렬해 종류별 TOP 3를 보여 주는 서비스.

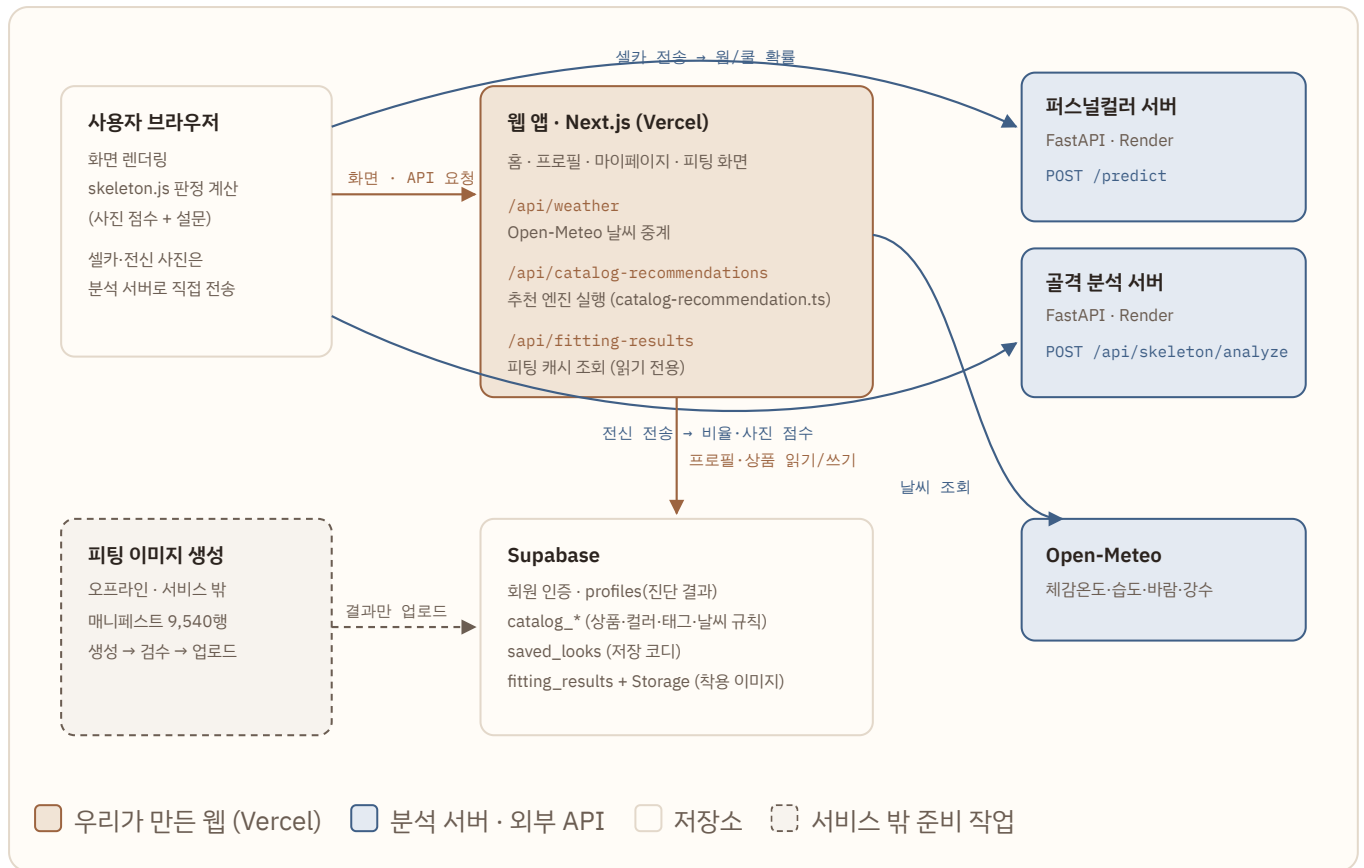
- 1 **입력** 지역, 상황(데일리·출근·데이트), 취향 설문 5문항, 셀카·전신 사진(선택)
- 2 **AI 진단** 셀카 → 웹·쿨 확률, 전신 + 설문 11문항 → 골격 3유형과 확신도
- 3 **날씨 필터** 체감온도·비 조건에 안 맞는 옷은 후보에서 제외
- 4 **점수 정렬** 날씨·컬러·상황·체형·실루엣·무드 점수를 더해 순위
- 5 **결과** 아우터·상의·하의·신발·가방별 TOP 3, 추천 이유, 가상 피팅

핵심 설계 원칙 3가지

- 1) **날씨가 먼저다.** 취향 점수가 아무리 높아도 날씨에 안 맞으면 아예 후보에서 빠진다. 점수 경쟁은 그다음이다.
- 2) **사진이 볼 수 있는 것만 사진에 맡긴다.** 골격 판정에서 정면 사진으로 알 수 없는 몸 두께는 설문으로 판단한다.
- 3) **사진은 저장하지 않는다.** 분석 요청에만 쓰고, 프로필에는 결과 값만 남긴다.

02 시스템 구조도

무엇이 어디서 실행되는지를 먼저 잡으면 나머지가 쉬워집니다.



꼭 기억할 포인트

사진은 웹 서버를 거치지 않고 브라우저에서 분석 서버로 바로 간다. 프로필 화면 코드가 `ondo-personal-color-ai.onrender.com/predict` 와 `ondo-skeleton-ai.onrender.com/api/skeleton/analyze` 를 직접 호출한다.

골격 최종 판정은 브라우저에서 계산된다. 서버는 사진 점수와 비율만 돌려주고, 설문과 합치는 2단계 판정은 `public/skeleton/skeleton.js` 의 `diagnose()` 가 한다.

추천은 웹 서버(Next.js API)에서 계산된다. 프로필은 로그인 세션으로 Supabase에서 읽고, 날씨 값은 화면이 주소 파라미터로 넘긴다.

구성 요소	저장소 위치	역할
웹 앱	app/ , components/ , lib/	화면, API 라우트, 추천 엔진
퍼스널컬러 서버	ondo_personal_color_api/service/	api.py(서버), run_skin_roi.py(피부 영역), 모델 2개
골격 서버	ondo_skeleton_api/	app.py(서버), skeleton_core.py(측정), params.json(기준값)
골격 판정	public/skeleton/	skeleton.js(2단계 판정), personas.json(시연 인물 69명)
DB 설계	supabase/migrations/	테이블 생성과 상품 시드 SQL
피팅 준비	ondo_virtual_tryon_api/ , scripts/	생성 노트북, 결과 업로드 스크립트

03 요청이 흐르는 길

사용자 행동 하나가 어떤 순서로 처리되는지 네 가지 장면으로 따라갑니다.

장면 1 · 셀카를 올린다

- 1 프로필 화면에서 셀카 선택 브라우저
- 2 이미지를 `POST /predict` 로 전송 브라우저 → 퍼스널컬러 서버
- 3 얼굴 랜드마크 → 피부 영역 → 색 특징 9개 → 로지스틱 회귀 퍼스널컬러 서버
- 4 `prediction`, `probabilities`, 평균 색 값을 응답. 사진은 메모리에서만 쓰고 버림
- 5 화면에 "웜 89%" 표시, 저장 시 결과만 profiles에 기록 브라우저 → Supabase

장면 2 · 전신 사진과 설문으로 골격 진단

- 1 전신 사진을 `POST /api/skeleton/analyze` 로 전송 브라우저 → 골격 서버
- 2 관절 검출, 6회 폭 측정, 비율 → 사진 점수 `p1`, 사진 근거 문장, 촬영 안내 골격 서버
- 3 설문 11문항 응답과 `p1` 을 `diagnose()` 에 넣어 2단계 판정 브라우저
- 4 유형, 확신도 %, 신뢰도, 근거 문장 표시 → 저장

장면 3 · 홈에서 오늘의 추천을 본다

- 1 지역으로 `/api/weather?city=busan` 호출 → Open-Meteo의 현재 체감온도·습도·바람·강수 웹 서버
- 2 상황과 날씨 값을 붙여 `/api/catalog-recommendations` 호출 브라우저 → 웹 서버
- 3 로그인 세션으로 profiles에서 퍼스널컬러·골격·취향 읽기, 상품·컬러·태그·날씨 규칙 전체 읽기 웹 서버 → Supabase
- 4 `recommendCatalogProducts()` 로 제외 → 점수 → 종류별 TOP 3 웹 서버

5 상품명·색상·점수·추천 이유를 화면에 표시

장면 4 · 가상 피팅을 연다

1 같은 추천 목록을 다시 불러와 상품 선택 [피팅 화면](#)

2 `/api/fitting-results?body=wave&tone=warm&variantId=...` 조회 [웹 서버 → Supabase](#)

3 캐시에 있으면 착용 이미지 표시, 없으면 "생성 캐시에 없습니다" 안내. 이 API는 이미지를 새로 만들지 않는다

04 퍼스널컬러 AI

셀카 한 장이 웬·쿨 확률이 되기까지의 다섯 단계와 그 안의 개념들.

- 1 **얼굴 랜드마크** MediaPipe Face Landmarker가 얼굴 위 478개 점의 좌표를 찾는다
- 2 **피부 영역(ROI)** 얼굴 외곽선 안쪽에서 눈·눈썹·입술 등을 지워 피부만 남긴 마스크를 만든다
- 3 **색 통계** 마스크 안 픽셀의 평균 색을 CIELAB, LCh, HSV 세 색공간 값으로 구한다 → 9개 숫자
- 4 **표준화** StandardScaler로 9개 값의 단위를 맞춘다 (평균 0, 표준편차 1)
- 5 **분류** 로지스틱 회귀가 웬일 확률과 쿨일 확률을 계산한다

개념 · 왜 RGB가 아니라 LAB인가

RGB는 화면이 빛을 섞는 방식이라 "노란기"나 "붉은기"가 숫자 하나로 드러나지 않는다. **CIELAB**은 사람 눈의 감각에 맞춘 색공간으로, **L***은 밝기, **a***는 초록(-)↔빨강(+), **b***는 파랑(-)↔노랑(+)
이다. 웬·쿨은 결국 피부의 노란기와 붉은기 균형이라 **b***가 핵심 단서가 된다.

LCh는 같은 LAB를 극좌표로 바꾼 것(**C** = 채도, **h** = 색상 각도), **HSV**는 색상·채도·명도다. 같은 색
을 세 관점에서 본 값을 함께 넣어 모델이 고르게 한다.

시연 아바타로 실제 돌린 결과: 웬 아바타 **b* 13.2** → 웬 74.2%, 쿨 아바타 **b* 7.0** → 쿨 99.7%.

개념 · 로지스틱 회귀

각 특징에 가중치를 곱해 더한 점수 **z**를 시그모이드 함수에 넣어 0~1 사이 확률로 바꾸는 분류 모델
이다. 가중치(계수)를 보면 어떤 색 값이 웬 쪽으로 밀었는지 설명할 수 있다.

$$z = w1 \cdot L^* + w2 \cdot a^* + w3 \cdot b^* + \dots + w9 \cdot V + b$$
$$P(\text{웬}) = 1 / (1 + e^{(-z)}) \quad \rightarrow 0.5\text{보다 크면 웬}$$

학습과 검증

생성형 AI 얼굴 205장(웬 103, 쿨 102)을 **8:2 층화 분할**해 164장으로 학습하고 41장으로 시험했다. 층
화는 학습·시험 양쪽에 웬·쿨 비율을 똑같이 맞추는 분할이다.

모델	학습 정확도	테스트 정확도	테스트 오분류
로지스틱 회귀 (채택)	98.2%	95.1% (39/41)	웜→쿨 1 · 쿨→웜 1
SVM (RBF 커널)	99.4%	95.1% (39/41)	웜→쿨 1 · 쿨→웜 1

테스트 성적이 같으면 학습 데이터에 덜 맞춰진(학습·테스트 차이가 작은) 모델이 처음 보는 사진에 더 안정적일 가능성이 크고, 계수로 설명도 된다. 그래서 로지스틱 회귀를 채택했다.

개념 · 혼동행렬

실제 정답(행)과 예측(열)을 교차해 센 표다. 대각선이 맞힌 수, 나머지가 틀린 방향이다. 웜 21장 중 20장, 쿨 20장 중 19장을 맞혔다.

05 골격 AI · 사진 측정

전신 사진 한 장에서 "사진 점수" p1 하나를 뽑아내는 과정. 코드는 `skeleton_core.py`.

- 1 **관절 검출** MediaPipe Pose(heavy 모델)로 33개 관절점. 이 중 어깨 11·12, 골반 23·24, 무릎 25·26, 발목 27·28번을 쓴다
- 2 **실루엣 마스크** 같은 모델이 내는 인물 분할 마스크에서 확률 0.5 이상을 몸으로 본다
- 3 **팔 지우기** 팔꿈치→손목→손끝 선을 굵기 50px(세로 1500px 기준)로 지운다. 손이 몸에 붙으면 골반 폭이 약 29% 부풀려지는 문제를 막기 위해서다
- 4 **폭 측정** 어깨 높이에서 어깨 폭, 골반 높이에서 골반 폭, 어깨~골반 사이 35~95% 구간에서 가장 좁은 줄을 허리 폭, 골반~무릎 25% 지점을 허벅지 폭으로 잰다
- 5 **6회 평균** 세로 1300·1500·1700px × 원본·좌우반전 = 6번 재서 평균. 흔들림을 $\pm 9\%$ p에서 $\pm 4\%$ p로 줄였다
- 6 **살집 보정** 무릎 폭/키 비율로 체형을 슬림·중간·살집으로 나누고, 살집이 많은 사진은 비율을 기울기만큼 보정한다
- 7 **표준점수(z)** 각 비율을 기준 평균·표준편차로 나눠 -1~+1로 자른다
- 8 **사진 점수 p1** 세 축을 가중 평균한다

$z(\text{비율}) = \text{clip}((\text{값} - \text{평균}) / \text{표준편차}, -1, +1)$

<code>center = z(어깨/골반)</code>	평균 1.165, 표준편차 0.028
<code>waist = z(허리/골반)</code>	평균 0.769, 표준편차 0.064
<code>frame = z(어깨/키)</code>	평균 0.230, 표준편차 0.007

$p1 = (20 \cdot (-center) + 20 \cdot (-waist) + 10 \cdot t \cdot (-frame)) / (20 + 20 + 10 \cdot t)$
 $t = \text{프레임 신뢰도} (\text{슬림 } 1.0 \cdot \text{중간 } 0.6 \cdot \text{살집 } 0.3)$

해석 · 왜 마이너스를 붙이나

웨이브의 특징은 어깨가 골반보다 좁고(center -), 허리가 잘록하고(waist -), 프레임이 가녀린 것(frame -)이다. 부호를 뒤집어 더하면 웨이브다울수록 p1 이 +1에 가까워진다. 살집이 많으면 어깨 폭 측정이 덜 믿을 만해서 frame의 비중(t)을 낮춘다.

실제 예시	어깨/골반	허리/골반	p1	사진 %
웨이브 예시 (v4_wave_02)	1.059	0.642	+0.814	91% 웨이브 쪽
스트레이트 예시 (v4_straight_01)	1.122	0.827	-0.169	42% 웨이브 아님

사진 근거 문장은 축 값이 ± 0.3 을 넘을 때 붙는다. 예를 들어 $\text{center} \leq -0.3$ 이면 "골반 대비 어깨가 좁은 하체 중심 체형", $\text{waist} \leq -0.3$ 이면 "허리가 잘록한 곡선형".

06 골격 AI · 2단계 판정

사진 점수와 설문을 합쳐 최종 유형을 정한다. 코드는 브라우저에서 도는 `skeleton.js` 의 `diagnose()`.

설문 응답 코드

11문항 각각 보기 4개: 1 스트레이트 보기, 2 웨이브 보기, 3 내추럴 보기, 4 잘 모르겠다. 문항마다 1단계용 가중치(W1)와 2단계용 가중치(W2)가 다르다.

1단계 · 웨이브인가?

```
q1 = Σ W1[문항] × (웨이브 보기면 +1, 아니면 -1) / Σ W1    ('모르겠다'는 제외)
w1 = 0.5 × p1 + 0.5 × q1
w1 ≥ 0.10 이면 웨이브
```

화면의 %는 이 점수를 바꾼 값이다: $\% = (\text{점수} + 1) \times 50$. 그래서 $w1 \geq 0.10$ 은 **55% 이상**과 같다. "웨이브 83%"는 $w1 = 0.66$ 이라는 뜻이다.

2단계 · 스트레이트 vs 내추럴 (웨이브가 아닐 때)

```
sn = Σ W2[문항] × (스트레이트 보기 +1, 내추럴 보기 -1) / Σ W2
|sn| < 0.10 이면 판정 보류, sn > 0 이면 스트레이트, sn < 0 이면 내추럴
```

2단계에는 사진을 넣지 않는다. 몸의 두께감과 뼈의 질감은 정면 사진으로 보이지 않기 때문이다.

확신도와 신뢰도

값	기준	뜻
최종 %	웨이브: 1단계 %. 그 외: 1단계 '웨이브 아님' %와 2단계 % 중 작은 값	가장 약한 단계가 전체 확신도를 정한다
신뢰도 높음	점수 ≥ 0.40 (70%)	
신뢰도 보통	점수 ≥ 0.25 (62.5%)	
신뢰도 낮음	그 미만, 또는 사진과 설문이 반대 방향	예: 사진은 하체 중심인데 설문은 상체 중심 → 이유 문장 표시

판정 기준의 근거

골격진단은 공개된 수치 기준이 없다. 골격진단 협회의 3유형 체계로 측정 항목·설문·추천 규칙을 정하고, Size Korea 인체치수조사를 참고하되 경계값은 데이터 분포로 정했다. 단일 이미지 체형 분류 선행연구(Trotter 외, 2023)의 정확도는 71.4%였다.

검증 (사진만으로 1단계)	웨이브	스트레이트	내추럴	전체
기존 60장	16/20	12/20	18/20	46/60
새 9장	3/3	3/3	3/3	9/9
합계	19/23	15/23	21/23	55/69 · 79.7%

설문까지 합친 정확도는 생성 이미지에 실제 설문 응답이 없어 측정할 수 없다. 정답대로 채운 설문에서 문항 일부를 무작위로 바꾸면 4문항 98.0%, 6문항 94.5%로 판정이 유지된다(강건성 시뮬레이션).

07 추천 엔진

`lib/catalog-recommendation.ts` 의 `recommendCatalogProducts()` . 상품 265개 × 컬러 6개 = 1,590개 후보를 매번 전부 계산한다.

1단계 · 제외 (하드 필터)

- 1 상품의 권장 체감온도 범위에서 **2°C 넘게** 벗어나면 제외
- 2 현재 강수량 2mm 이상인데 비에 안 맞는 상품이면 제외 (강수 확률이 아니라 실제 강수량 기준)
- 3 체감 24°C 이상이면 아우터 전부 제외
- 4 더운 날 맞지 않는 소재 제외: 16°C 이상 두꺼운 코트·패딩, 25°C 이상 니트·맨투맨류, 건조한 16°C 이상 부츠 등

2단계 · 점수

항목	범위	계산
날씨	0 ~ 50	온도 범위 안 30 ($\pm 2^{\circ}\text{C}$ 경계 14) + 습도 6 + 바람 5 + 강수 6 + 일교차 3
퍼스널컬러	0 ~ 20	내 톤과 같은 색 20, 웜·쿨 공용 12, 진단 없음 8, 다른 톤 0
상황	± 18	상품의 상황 태그 점수(-3~+3)를 18점 만점으로 환산
체형	± 15	같은 방식, 골격 유형 태그
실루엣	± 10	취향 설문의 선호 실루엣 태그
무드	± 4	취향 설문의 무드 태그

태그 점수 = `clip( 태그값 / 3 × 만점 , -만점 , +만점 )`
총점 = 날씨 + 컬러 + 상황 + 체형 + 실루엣 + 무드

- 1 종류(아우터·상의·하의·신발·가방)별로 총점 내림차순 정렬
- 2 같은 상품의 다른 색은 한 번만 남긴다 (컬러 중복 제거)

3 점수가 같으면 날짜와 상품 ID로 만든 해시로 순서를 정한다. 그래서 같은 날에는 순서가 고정되고, 날이 바뀌면 동점 상품 순서가 바뀔 수 있다

4 상위 3개와 추천 이유("웜 톤 색상", "체감 18° 적합", "웨이브 체형 가점")를 반환

튜닝 이력

처음에는 상황 가중치가 ± 6 이라 데일리·출근·데이트를 바꿔도 추천이 거의 같았다. ± 18 로 세 배 올려 상황이 순위를 실제로 움직이게 했다. 웜·웨이브, 체감 18°C에서 출근을 고르면 1위가 트러커 자켓→숏 트렌치, 니트 탑→린넨 셔츠, 메리제인→페니 로퍼, 솔더백→사첼백으로 바뀐다.

08 데이터베이스 (Supabase)

서비스에서 실제로 쓰는 테이블 위주로 정리했습니다.

테이블	담는 것	누가 쓰나
profiles	퍼스널컬러·골격 결과(AI 결과와 출처 구분), 취향, 지역	프로필 저장, 추천 API가 읽음
catalog_products	상품 265개: 종류, 세부 유형, 핏, 소재, 보온·통기 등급	추천 API
catalog_variants	컬러 1,590개: 색 이름, 이미지 경로, 웜/쿨/공용	추천 API
catalog_tags	체형·무드·실루엣·상황 태그와 -3~+3 점수	추천 API
catalog_weather_rules	상품별 권장 체감온도, 습도·바람 한도, 강수 허용, 일교차	추천 API의 제외·날씨 점수
saved_looks	저장한 코디와 당시 날씨	마이페이지
fitting_results	체형·톤·컬러별 착용 이미지 주소와 상태	피팅 캐시 조회 API

상품 데이터는 0008_catalog_seed.sql 로 한 번에 들어가고, 상황 점수는 0009·0010 마이그레이션에서 다시 매겼다. 상품·태그 테이블은 누구나 읽을 수 있고, 프로필과 저장 코디는 본인만 읽고 쓸 수 있게 행 단위 보안(RLS)이 걸려 있다.

09 가상 피팅 파이프라인

실시간 생성이 아니라 "미리 만들어 두고 꺼내 쓰는" 구조.

- 1 매니페스트 컬러 1,590 × 아바타 6(웜·쿨 × 스트레이트·웨이브·내추럴) = 9,540행. 행마다 `job_id = 체형_톤_컬러ID`
- 2 생성 아바타 이미지에 상품 한 개씩 입힌 이미지를 생성 (서비스 밖 작업)
- 3 검수 얼굴·체형 보존, 손잡이 구조, 투명 배경 등을 확인하고 승인
- 4 업로드 승인된 이미지를 Storage에 올리고 `fitting_results`에 등록 (관리자 키는 로컬에서만)
- 5 조회 피팅 화면이 체형·톤·컬러ID로 캐시를 찾고, 없으면 안내 문구

왜 이렇게 만들었나

무료 GPU는 하루 사용량이 짧고 대기열이 길어, 사용자가 누를 때마다 생성하면 시연이 멈춘다. 미리 만들어 두면 GPU가 꺼져 있어도 이미 만든 결과는 항상 보인다. 대신 9,540장을 다 만들려면 시간이 들어, 시연 시나리오에서 추천 엔진이 실제로 띄우는 상품(웜·웨이브 기준 54개)부터 생성했다.

10 개인정보 설계

- 1 학습·검증·시연 이미지는 모두 생성형 AI 이미지. 실제 인물 사진 없음
- 2 사용자 사진은 분석 요청에만 쓰고 원본·경로를 저장하지 않음
- 3 프로필에는 최종 판정, 설문 코드, 취향 값만 저장
- 4 피팅 이미지는 사용자 사진이 아니라 아바타로 생성
- 5 Supabase 키는 로컬 `.env.local`에만 두고 저장소에 올리지 않음

11 핵심 숫자 한 장

95.1%

퍼스널컬러 테스트 정확도 (39/41)

79.7%

골격 사진만의 웨이브 판별 (55/69)

55%

웨이브 판정 기준선 (점수 0.10)

50 : 50

1단계 사진 : 설문 비중

6회

사진 한 장 측정 횟수

$\pm 2^{\circ}\text{C}$

온도 범위 허용 오차

± 18

상황 가중치 (기준 ± 6)

9,540

가상 피팅 전체 조합

12 용어집

ROI

Region of Interest. 분석할 영역. 여기서는 얼굴에서 피부만 남긴 부분

랜드마크

얼굴이나 몸의 기준점 좌표. MediaPipe가 얼굴 478개, 몸 33개를 찾는다

세그멘테이션 마스크

픽셀마다 사람인지 배경인지 표시한 흑백 지도

CIELAB

밝기 L^* , 빨강-초록 a^* , 노랑-파랑 b^* 로 색을 표현하는 색공간

표준화

평균을 빼고 표준편차로 나눠 서로 다른 단위의 값을 같은 척도로 맞추는 것

z점수

값이 평균에서 표준편차 몇 개만큼 떨어졌는지. 여기서는 ± 1 로 자름

로지스틱 회귀

특징의 가중합을 시그모이드로 확률로 바꾸는 분류 모델

SVM

두 클래스 사이 경계를 가장 넓게 긋는 분류 모델. RBF 커널로 곡선 경계 가능

층화 분할

학습·시험 세트에 클래스 비율을 똑같이 유지하며 나누는 방법

과적합

학습 데이터에만 지나치게 맞춰져 새 데이터에서 성능이 떨어지는 상태

혼동행렬

실제 정답과 예측을 교차해 센 표

확신도

ONDO에서 점수 $-1 \sim +1$ 을 $0 \sim 100\%$ 로 옮긴 값. 확률이 아님

하드 필터

점수와 상관없이 조건에 안 맞으면 후보에서 빼는 규칙

RLS

Row Level Security. 행 단위로 누가 읽고 쓸 수 있는지 정하는 DB 보안 규칙

매니페스트

만들어야 할 결과물 전체 목록. 피팅은 9,540행

캐시

미리 만들어 저장해 둔 결과. 요청 시 새로 계산하지 않고 꺼내 쓴다

13 셀프 체크

답을 떠올린 뒤 펼쳐서 확인하세요.

Q 사용자 셀카는 웹 서버(Vercel)를 거치나?

아니다. 브라우저가 Render의 퍼스널컬러 서버 `/predict` 로 바로 보낸다. 웹 서버는 결과 저장과 추천만 맡는다.

Q "웨이브 83%"를 점수로 바꾸면 얼마이고, 확률인가?

$(83 \div 50) - 1 = 0.66$. 사진 점수와 설문 점수의 평균을 %로 옮긴 확신도이며 확률이 아니다.

Q 2단계(스트레이트 vs 내추럴)에 사진을 쓰지 않는 이유는?

두 유형의 차이는 몸의 두께감과 뼈의 질감인데, 정면 사진의 폭 측정으로는 보이지 않기 때문이다.

Q p1 공식에서 center·waist·frame 앞에 마이너스가 붙는 이유는?

웨이브는 어깨가 좁고 허리가 잘록하고 프레임이 가녀려서 세 값이 모두 음수 쪽이다. 부호를 뒤집어야 웨이브다울수록 p1이 커진다.

Q SVM과 로지스틱 회귀의 테스트 정확도가 같은데 로지스틱을 고른 이유는?

SVM은 학습 정확도만 99.4%로 더 높아 과적합 위험이 있고, 로지스틱은 학습·테스트 차이가 작으며 계수로 판단 근거를 설명할 수 있다.

Q 체감 24°C인 날 아우터가 추천에 나오나?

안 나온다. 24°C 이상에서는 아우터가 하드 필터로 전부 제외된다.

Q 추천 점수에서 가장 비중이 큰 항목은?

날씨(최대 50). 그다음 퍼스널컬러 20, 상황 ± 18 , 체형 ± 15 순이다. 그리고 날씨는 점수 이전에 제외 규칙으로도 작동한다.

Q 가상 피팅 화면에서 "생성 캐시에 없습니다"가 뜨는 이유는?

해당 체형·톤·컬러 조합의 착용 이미지가 아직 생성·승인되지 않아 fitting_results에 없기 때문이다. 피팅 API는 이미지를 새로 만들지 않는다.

Q 같은 조건인데 어제와 오늘 추천 순서가 조금 다른 이유는?

점수가 같은 상품의 순서를 날짜가 들어간 해시로 정하기 때문이다. 같은 날에는 고정된다.